

FitTrack AI

Personalized Fitness Recommendations Powered by AI

Phase 3 – Project Design

Project type: AI-powered fitness tracking and recommendation REST API / backend

Stack: Node.js · Express.js · MongoDB · Mongoose · JWT · bcrypt.js · Google Gemini AI · Thunder Client / Postman

1. Technical Architecture

FitTrack AI follows a layered (n-tier) REST architecture. Each request passes through a fixed chain of layers, and every layer has one job. This keeps the code easy to test, debug and extend. AI requests take an additional branch from the controller layer to Google Gemini.

1.1 Request Flow

Standard flow: Client / API Testing Tool → Express.js Server → API Routes → Authentication Middleware → Controllers → Services → Mongoose Models → MongoDB Atlas.

AI flow: Controllers → Gemini AI Service → Google Gemini API → AI Recommendation / Fitness Insight.

1.2 Purpose of Each Component

Client / API Testing Tool	Thunder Client or Postman sends HTTP requests (with JSON bodies and the JWT in the <u>Authorization</u> header) and displays the JSON responses.
Express.js Server	Entry point of the backend. Loads environment variables, connects to MongoDB, applies global middleware (JSON body parsing, CORS) and mounts the routes.

API Routes	Map each URL and HTTP method (e.g. <code>POST /api/workouts</code>) to the correct controller function and attach middleware to protected endpoints.
Authentication Middleware	Reads the Bearer token, verifies it with the JWT secret, loads the user and attaches it to <code>req.user</code> . Requests without a valid token are rejected with 401.
Controllers	Handle the request and response cycle: read input, run validation, call services, and return standard JSON responses.
Services	Hold the business logic (password hashing, token creation, workout queries, ownership checks, search filters), so controllers stay thin.
Gemini AI Service	Builds prompts from the user's workout data, calls the Gemini API, and converts the model output into a clean recommendation or insight.
Mongoose Models	Define the <code>User</code> and <code>Workout</code> schemas, validation rules, timestamps and the <code>Workout</code> → <code>User</code> reference.
MongoDB Atlas	Cloud-hosted database that persistently stores users and workouts.
Google Gemini API	External generative AI that produces personalized workout recommendations and fitness insights.

FitTrack AI – Technical Architecture Diagram

Client / Thunder Client or Postman
 Sends HTTP requests + JWT, shows JSON
 Express.js Server
 JSON parser · CORS · env config · DB connect
 API Routes
`/api/auth` · `/api/workouts` · `/api/ai`
 Authentication Middleware
 Verify JWT · attach `req.user`
 Controllers
 Validate input · call services · send response
 Services
 Business logic · hashing · queries · search
 Mongoose Models
`User` · `Workout` schemas
 MongoDB Atlas
 Cloud database (users, workouts)
 HTTP request route match
 protected routes authorized request
 business call data access queries (Mongoose driver)
 AI LAYER
 Gemini AI Service
 Build prompt · call API · parse output
 Google Gemini API
 External generative AI model
 AI Recommendation / Fitness Insight
 Returned to controller as JSON
 AI request prompt response
 AI result returns to controller
 Legend
 Client layer API / server layer

Authentication layer Business logic layer Database layer AI layer Figure 1: FitTrack AI – Technical Architecture Diagram (solid arrows: request flow; dashed arrow: AI result returning to the controller)

2. ER Diagram

The database has two collections: **User** and **Workout**. No other entities exist in the project.

2.1 Relationship

One User can have many Workouts (**USER 1 — N WORKOUT**). Each Workout belongs to exactly one User. The `Workout.user` field stores the `_id` of the owning User (a Mongoose `ObjectId` with `ref: "User"`). This reference is what restricts every user to viewing and changing only their own workouts.

2.2 Entity Definitions

USER	<code>_id</code>	PK	Auto-generated ObjectId
	<code>name</code>		Required string
	<code>email</code>		Required, unique
	<code>password</code>		Stored as bcrypt hash
	<code>createdAt</code>		Timestamp
	<code>updatedAt</code>		Timestamp
WORKOUT	<code>_id</code>	PK	Auto-generated ObjectId

workoutName		Required string
category		String (e.g. Cardio, Strength)
duration		Number (minutes)
caloriesBurned		Number
workoutDate		Date
user	FK	ObjectId, ref → User._id
createdAt		Timestamp
updatedAt		Timestamp

FitTrack AI – Entity Relationship Diagram
 USER _idPK name : String email : String (unique) password : String (hashed) createdAt : Date updatedAt : Date
 WORKOUT _idPK workoutName : String category : String duration : Number caloriesBurned : Number workoutDate : Date user : ObjectIdFK → USER._id createdAt : Date updatedAt : Date
 has many 1 N Legend PK = Primary Key (_id) FK = Foreign Key / Reference (Workout.user → User._id) Cardinality: USER 1 — N WORKOUT
 (one user has many workouts; each workout has exactly one user)
 Figure 2: FitTrack AI – Entity Relationship Diagram

3. Features

The following features make up the complete scope of FitTrack AI. Endpoint paths shown are the planned design and may be adjusted during implementation.

3.1 Feature Summary Table

4. Roles and Responsibilities

4.1 System User Role: Authenticated User

FitTrack AI has a single user role. Anyone can register and log in; once authenticated with a valid JWT, the user can perform the actions below on their *own* data only. No Admin role is implemented.

- Register an account and log in
- Access the protected profile
- Add, view, search, update and delete workouts
- Request an AI workout recommendation
- Request AI fitness insights

4.2 Component and Role Responsibilities

5. User Flow

The flow below traces a typical journey of an authenticated user, from first visit to receiving a response. Your prompt ended at this section, so the flow is derived from the features in Section 3.

5.1 Step-by-Step Flow

1. The user opens Thunder Client or Postman and decides whether they already have an account.
2. A new user calls the registration endpoint. The server validates the input, hashes the password with bcrypt.js and stores the user in MongoDB.
3. The user logs in with email and password. On success the server returns a signed JWT; on failure it returns a 401 error.
4. The user sends the token in the `Authorization: Bearer <token>` header with every protected request.
5. The authentication middleware verifies the token. Invalid or missing tokens are rejected with 401.
6. The user chooses an action: view profile, manage workouts (add, view, view by ID, update, delete), search workouts, or request AI recommendation / AI insights.
7. The controller validates input and calls the service layer. Workout actions go through Mongoose models to MongoDB Atlas. AI actions go through the Gemini AI Service to the Google Gemini API.
8. Any error is handled by the centralized error handler; every response, successful or not, is returned as standard JSON.

FitTrack AI – User Flow Diagram Start Has account? No Register validate · hash password · save user Yes Login (email + password) Valid credentials? No 401 error JSON response Yes Receive JWT token Send request with Authorization: Bearer token Token valid? No 401 Unauthorized Yes Choose an action View profile · Add / View / Update / Delete workout Search workouts · AI recommendation · AI insights Controller → ServiceMongoose → MongoDB Atlas Gemini AI Service → Google Gemini API Standard JSON response → End Errors at any stage are caught by the centralized error handler and returned as JSON. Figure 3: FitTrack AI – User Flow Diagram